First Steps in a Paraconsistent Transition Systems Toolkit

Rodrigo Alves[1], Juliana Cunha[1,2,3(✉)], and Alexandre Madeira[1,2]

[1] Department Mathematics, Aveiro University, Aveiro, Portugal
{rodrigoalves,juliana.cunha,madeira}@ua.pt
[2] CIDMA - Center for Research and Development in Mathematics and Applications, Aveiro, Portugal
[3] INESC TEC, Department of Informatics, Minho University, Braga, Portugal

Abstract. Modelling complex information systems (e.g., combinations of heterogeneous data sources, data from hybrid-system sensors, or medical diagnostic exams) often requires managing inconsistencies in which different decisions and instructions may simultaneously align or conflict. Decision and reasoning methods for such scenarios are a focus of paraconsistent logics, which provide a means of handling inconsistencies without collapsing into triviality, treating them as potentially informative rather than anomalous.

Paraconsistent labelled transition systems (PLTS) were introduced as transition structures– parameterised by an underlying lattice– where each transition is characterised by a pair of values: one representing evidence that the transition occurs and the other representing evidence against it. Together with their associated modal logic and process algebra, PLTS provide a formal framework for modelling and reasoning in inconsistent scenarios. In this paper, we present an interactive toolkit that allows users to construct and visualise PLTS, as well as to interpret modal formulae over them. Finally, illustrative examples of the applicability of the framework and the toolkit are explored in fields such as robotics and finance.

Keywords: Paraconsistency · Transition Systems · Inconsistencies

1 Introduction

Modelling complex systems often entails managing inconsistencies, where different decisions or instructions may simultaneously align or conflict. Moreover, the synthesis of transition systems from input data is central to many applications. In such settings, input data is frequently incomplete or even contradictory, especially when collected from multiple, possibly unreliable, sources. For

This work is supported by CIDMA (https://ror.org/05pm2mw36) under the Portuguese Foundation for Science and Technology (https://ror.org/00snfqn58), Grants https://doi.org/10.54499/UID/04106/2025 (UID/04106/2025 and UID/PRR/04106/2025), and with project Banksy COMPETE2030-FEDER-00892000. Cunha is also supported by the FCT PhD Grant 2024.04652.BD.

G. De Giacomo et al. (Eds.): TASE 2026, LNCS 16697, pp. 368–385, 2027.
https://doi.org/10.1007/978-3-032-30693-7_24

example, during a development process, engineering teams often make design decisions based on imprecise and sometimes contradictory requirements elicited from stakeholders with distinct perspectives. Another example arises in robotics, where a robot collecting information through sensors may face inconsistencies, since sensors are sensitive to external conditions. Similar situations also occur in finance, where individuals may rely on different metrics, sometimes contradictory, to assess whether buying or selling a stock is profitable. Consequently, the development of more flexible and robust models for reasoning about systems in environments with potentially conflicting information is becoming increasingly relevant across different contexts.

Motivated by such challenges, paraconsistent labelled transition systems, in short PLTS, together with their associated modal logic, were introduced in [7]. They provide a way to model such situations through paraconsistent reasoning capable of capturing ignorance, truth, falsity, and contradiction.

PLTS and their modal logic follow a parametric approach, in line with works such as [5,18], allowing the framework to support a wider range of application scenarios. Thus, all relevant constructions are parametric over a class of lattices, allowing different instantiations depending on the structure of truth values most suitable for a given problem. More precisely, the framework underlying this paper starts from a complete residuated lattice $\mathbf{A} = (A, \leqslant)$ and constructs the product of $\mathbf{A}$ with its order dual $\mathbf{A}^{\partial}$. The resulting structure, known as a twist structure, a term coined by [17], plays a key role in PLTS research by providing a systematic way to combine pairs of values in A^2. A PLTS is then defined as a transition system whose valuations and transition relations take values in A^2. Informally, a pair $(\mathit{tt}, \mathit{ff}) \in A^2$ can be interpreted such that tt represents the amount of information supporting a statement, while ff represents the amount of information against it. Consequently, this framework accommodates a wide range of epistemic scenarios, since the two values are not required to be complementary. For instance, if the underlying lattice is the Boolean algebra, the four pairs $(1,0)$, $(0,1)$, $(1,1)$, and $(0,0)$ correspond respectively to truth (only evidence in favor of a statement), falsity (only evidence against a statement), inconsistency (simultaneous evidence both for and against), and vagueness (neither evidence for nor against). Thereby yielding the well-known Belnap-Dunn lattice [4].

Furthermore, in this paper we consider the paraconsistent modal logic introduced in [7], which supports reasoning about PLTS. Paraconsistent logics emerged as a way to reason in the presence of inconsistencies without collapsing into triviality [20]. They have found applications in engineering [2], robotics [1], quantum logic [6], and quantum computing [3]. While the motivations for their use vary, these logics share the fundamental view that inconsistency is not merely an artefact of error or confusion; rather, it may be informative and support meaningful reasoning.

Prior work on PLTS and their corresponding modal logic has been explored from an institution-theoretic perspective in [8,10], following the stepwise implementation methodology outlined by Sannella and Tarlecki. PLTS have also been

studied in [9] in the context of reactive graphs, where transitions may affect others and policies are introduced to resolve such conflicts. Finally, potential applications of PLTS in quantum computing have been explored in [3] to model qubit decoherence in quantum circuits that, due to environmental interference, do not behave according to their intended design.

As is well known, a formal method becomes effective in practice only when it is supported by appropriate tools. Moreover, the availability of user-friendly interfaces is essential to broaden the audience of practitioners who can benefit from the formalism. This paper contributes in this direction by introducing a toolkit[1] for the development of PLTS. Namely, by allowing users to:

(i) construct arbitrary finite complete residuated lattices and automatically generate their twist structures;
(ii) build and visualise PLTS parametric over a chosen lattice; and
(iii) interpret modal formulae and check their validity over PLTS using the semantics introduced in [7].

Additionally, this paper explores case studies to illustrate the potential applications of the framework and the proposed toolkit in areas such as robotics and finance. Accordingly, the presented toolkit is expected to play a key role in future applications of the ongoing research agenda on twist structures and PLTS [7–10], aimed at better understanding and reasoning in the presence of inconsistencies.
Structure of the Paper: Section 2 recalls the necessary background from [7, 10]. Section 3 presents the proposed toolkit, its main features, and illustrative applications. Concluding remarks appear in Sect. 4.

2 Paraconsistent Labelled Transition Systems and Their Modal Logic

This section introduces the parametric framework employed throughout the paper and recalls the necessary background from [7,10].

As in other works [5,18], the approach taken is parametric over a class of lattices, accommodating different instances according to the structure that best suits each concrete application. More precisely, each complete lattice $\mathbf{A} = (A, \leqslant)$ acts over a non-empty set A of finite truth values, bounded by 0 and 1, equipped with binary operations $\sqcap$ (meet) and $\sqcup$ (join) satisfying the usual distributive laws. The adjunction property is stated as $a \sqcap b \leqslant c$ iff $b \leqslant a \rightarrow c$, where $\rightarrow$ denotes the implication operation of $\mathbf{A}$.

Example 1. Well-known examples of complete distributive residuated lattices used throughout this paper include:

– The two element Boolean algebra $\mathbf{2} = (\{0, 1\}, \leqslant_{\mathbf{2}})$ with $0 \leqslant_{\mathbf{2}} 1$, and the meet and join operations are standard Boolean conjunction and disjunction.

[1] Available at https://github.com/juliana-cunha/PLTS_toolkit.git.

– The Łukasiewicz three-valued algebra $\mathbf{3} = (\{0, 1/2, 1\}, \leqslant_{\mathbf{3}})$ with $0 \leqslant_{\mathbf{3}} 1/2 \leqslant_{\mathbf{3}}$ and operations defined as follows:

$\sqcap_{\mathbf{3}}$	0	1/2	1
0	0	0	0
1/2	0	1/2	1/2
1	0	1/2	1

$\sqcup_{\mathbf{3}}$	0	1/2	1
0	0	1/2	1
1/2	1/2	1/2	1
1	1	1	1

$\rightarrow_{\mathbf{3}}$	0	1/2	1
0	1	1	1
1/2	1/2	1	1
1	0	1/2	1

The intermediate truth value $1/2$, sometimes denoted by u or $\#$, stands for "possibility", "unknown" or "indeterminacy", respectively c.f. [20].

As stated, this framework is extended by working with pairs $(\#, \not\#) \in A^2$, parametric over $\mathbf{A}$, where $\#$ captures evidence in favor of a statement, and $\not\#$ evidence against it. Since these values need not be complementary, this allows the representation of (i) vague information, when the values sum to less than 1; (ii) consistent information, when they are complementary and sum to 1; and (iii) inconsistent information, when they sum to more than 1. Such pairs can be constructed as the product of the lattice $\mathbf{A}$ with its order dual $\mathbf{A}^{\partial}$. The resulting structure forms a twist structure [17], which provides a systematic way to combine pairs of values in A^2. Formally,

Definition 1. *Let $\mathbf{A} = (A, \leqslant)$ be a complete distributive residuated lattice. The $\mathbf{A}$-twist structure is $(A^2, \leqslant_t)$, where for every $x_1, x_2, y_1, y_2 \in A$,*

$$
\begin{gathered}
/\!\!/\,(x_1, y_1) = (y_1, x_1) \\
(x_1, y_1) \leqslant_t (x_2, y_2) \text{ iff } x_1 \leqslant x_2 \text{ and } y_1 \geqslant y_2 \\
(x_1, y_1) \sqcap\!\!\sqcap (x_2, y_2) = (x_1 \sqcap x_2, y_1 \sqcup y_2) \\
(x_1, y_1) \sqcup\!\!\sqcup (x_2, y_2) = (x_1 \sqcup x_2, y_1 \sqcap y_2) \\
(x_1, y_1) \odot (x_2, y_2) = (x_1 \sqcap x_2, x_1 \rightarrow y_2 \sqcap x_2 \rightarrow y_1) \\
(x_1, y_1) \Rightarrow (x_2, y_2) = (x_1 \rightarrow x_2 \sqcap y_2 \rightarrow y_1, x_1 \sqcap y_2)
\end{gathered}
$$

Therefore, since $(A^2, \leqslant_t)$ is the product of two complete distributive lattices, it is itself a complete distributive lattice. Moreover, operator $\Rightarrow$ is the right adjoint of $\odot$ (cf. [7]), that is, for any (x_1, y_1), (x_2, y_2), $(x_3, y_3) \in A^2$:

$$
(x_1, y_1) \odot (x_2, y_2) \leqslant_t (x_3, y_3) \text{ iff } (x_1, y_1) \leqslant_t (x_2, y_2) \Rightarrow (x_3, y_3)
$$

Thus, it follows that $(A^2, \leqslant_t)$ is itself a complete distributive residuated lattice.

Informally, a pair in A^2 is said to be more truthful when there is more evidence in favor and less against. Operations $\sqcap\!\!\sqcap$ and $\sqcup\!\!\sqcup$ correspond respectively to the meet and join under $\leqslant_t$ [13] and $/\!\!/$ is an involutive negation that reverses truth [11], that is, if $(x_1, y_1) \leqslant_t (x_2, y_2)$, then $/\!\!/(x_1, y_1) \geqslant_t /\!\!/(x_2, y_2)$.

The following example illustrates the twist structures induced by the lattices of Example 1.

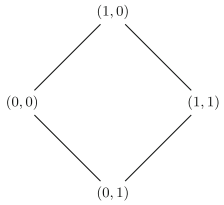

Fig. 1. Lattice $\mathcal{FOUR}$.

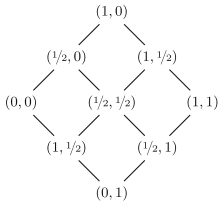

Fig. 2. Lattice $\mathcal{NINE}$.

Example 2. The twist structure over **2** is the lattice $\mathcal{FOUR}$, while that over **3** yields $\mathcal{NINE}$, whose Hasse diagrams are shown in Figs. 1 and 2, respectively.

Following [7,10], transitions and valuations in a paraconsistent transition system take values in A^2, parametrised by an underlying lattice $(A, \leqslant)$. We now recall the formal definition.

Definition 2 ([7]). *An* $(\mathbf{A}, Act, Prop)$*-paraconsistent transition system, in short PLTS, over a complete distributive residuated lattice* $(\mathbf{A}, \leqslant)$*, a set of actions Act and a set of proposition symbols Prop is a structure* (W, R, V) *where:*

- W *is a non-empty set of states;*
- $R : W \times Act \times W \to A^2$ *is a paraconsistent relation assigning, to each transition, evidence for and against its traversal; and*
- $V : W \times Prop \to A^2$ *is a valuation assigning, to each state and each proposition, the evidence for and against its holding.*

For readability, transitions weighted by $(0, 1)$, representing only evidence against the transition, are omitted from graphical representations. Additionally, throughout the paper, we assume that the transition function of any PLTS is complete; any undefined transition is assigned the default value $(0, 1)$.

We use the standard set of modal formulae, denoted by $\mathsf{Fm}(\mathrm{Act}, \mathrm{Prop})$, given by the following syntax: $\varphi \coloneqq \bot \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \langle a \rangle \varphi$, where $p \in \mathrm{Prop}$ and $a \in \mathrm{Act}$. Additionally, we consider the usual abbreviations from propositional logic: $\top = \neg\bot$, $\varphi \vee \varphi' = \neg(\neg\varphi \wedge \neg\varphi')$, $\varphi \rhd \varphi' = \neg(\varphi \wedge \neg\varphi')$, $\varphi \rhd\lhd \varphi' = (\varphi \rhd \varphi') \wedge (\varphi' \rhd \varphi)$ as well as from modal logic $[a]\varphi = \neg\langle a \rangle \neg\varphi$.

Given a PLTS $M = (W, R, V)$ and a formula $\varphi \in \mathsf{Fm}(\mathrm{Act}, \mathrm{Prop})$ the satisfaction relation $\models_M$ assigns to each state $w \in W$ a pair in A^2 capturing evidence for and against φ holding at that state. More precisely,

Definition 3 ([7]). *Let* $M = (W, R, V)$ *be an* $(\mathbf{A}, Act, Prop)$*-PLTS. The satisfaction relation* $\models_M : W \times \mathsf{Fm}(Act, Prop) \longrightarrow A^2$ *is defined recursively as follows:*

$$
\begin{aligned}
(w \models_M \bot) &= (0,1)\\
(w \models_M p) &= V(w,p)\\
(w \models_M \neg\varphi) &= /\!\!/(w \models_M \varphi)\\
(w \models_M \varphi \wedge \varphi') &= (w \models_M \varphi) \sqcap (w \models_M \varphi')\\
(w \models_M \langle a\rangle\varphi) &= \bigsqcup_{v\in W} (R_a(w,v) \odot (v \models_M \varphi))
\end{aligned}
$$

for $p \in Prop$ and $a \in Act$.

Note that since $\models_M$ returns a value in A^2, naturally it is defined using the operations of the twist structure in order to compute values in A^2.

One may then naturally extend the local satisfaction relation $\models_M$ to the global satisfaction relation:

$$(M \models \varphi) = \bigsqcap_{w\in W} (w \models_M \varphi) \tag{1}$$

Example 3. Let $M = (\{s_0, s_1, s_2\}, R, V)$ be a $(\mathbf{2}, \{a\}, \{p, q\})$-PLTS with transitions in R as depicted in Fig. 3 and the valuation given by $V(s_0, q) = V(s_2, q) = (1, 0)$, $V(s_2, p) = (1, 1)$ and $V(s_1, p) = V(s_1, q) = V(s_0, p) = (0, 1)$.

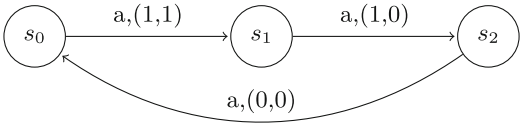

Fig. 3. The $(\mathbf{2}, a, p, q)$-PLTS M.

One may then compute $(s_i \models_M \neg p \vee q) = (1, 0)$ for $i \in \{0, 1, 2\}$, which indicates that there is only evidence in favor of M satisfying the formula. Moreover, $(s_j \models_M [a]p \vee q) = (1, 0)$ for $j \in \{0, 2\}$ while

$$
\begin{aligned}
(s_1 \models_M [a]p \vee q) &= (s_1 \models_M \neg\langle a\rangle\neg p \vee q)\\
&= (s_1 \models_M \neg\langle a\rangle\neg p) \sqcup (s_1 \models_M q)\\
&= /\!\!/ (R_a(s_1, s_2) \odot /\!\!/ V(s_2, p)) \sqcup V(s_1, q)\\
&= (0, 1)
\end{aligned}
$$

which indicates that although s_2 is reachable from s_1, at s_2 the proposition p holds inconsistently; in other words, in this framework classical truth $(1, 0)$ does not entail inconsistent truth $(1, 1)$.

3 A Toolkit for PLTS and the Associated Modal Logics

This section presents the implementation of the proposed toolkit for constructing and visualising PLTS as well as interpreting formulae over PLTS as defined in Sect. 2. Case studies are also provided to illustrate both the paraconsistent framework and the features of the toolkit.

The proposed toolkit, implemented in Python, is available at https://github.com/juliana-cunha/PLTS_toolkit.git. This open-source tool provides an interactive interface for generating twist structures, visualising PLTS, and interpreting modal formulae. It is built using `PyQt6` [21] graphical interface, with the core logic separating the mathematical definitions from the UI presentation layer. Additionally, model persistence is handled via `JSON`, enabling users to save and reload objects across sessions. Graphical rendering of Hasse diagrams and transition systems is supported through `Matplotlib` [16] and `NetworkX` [14].

The main window, shown in Fig. 4, follows a split-pane layout. The top menus provide the following actions: New, for creating objects; Load, for importing existing ones; Delete for remove objects; See for viewing previously saved JSON files; View, for switching between light and dark mode; and Help which provides mathematical definitions and a legend of the notation used. The left sidebar serves as a project explorer, grouping objects into Lattices, Residuated Lattices, Twist Structures, States, Models, and Propositions; the right panel displays object details, visualisations, and the modal logic interpreter.

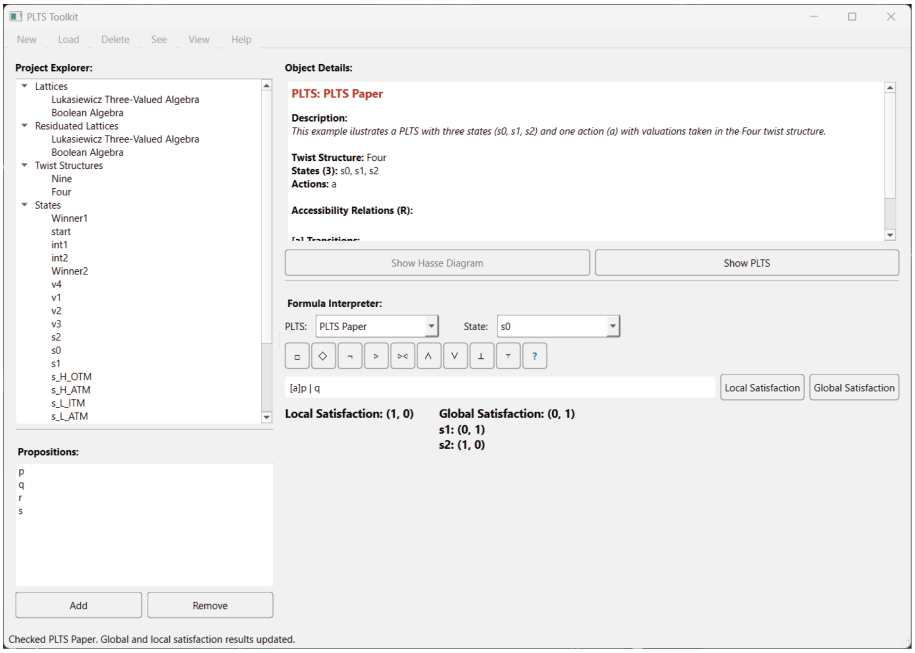

Fig. 4. Screenshot of the PLTS toolkit.

The toolkit provides built-in support for loading preexisting standard lattices, such as **2** and **3** of Example 1, via the Load menu. Custom finite lattices $(A, \leqslant)$ forming complete distributive residuated lattice can also be created via the New menu by specifying the set of truth values A, the partial order $\leqslant$, and optionally the lattice operations.

The toolkit also includes built-in twist structures, allowing users to load $\mathcal{FOUR}$ and $\mathcal{NINE}$ from Example 2. In addition, custom twist structures, defined as the product of a complete lattice with its order dual, can be generated automatically from any existing lattice. As expected, the size of the twist structure grows with the size of the underlying lattice. As noted, the paraconsistent framework is parametric over a class of lattices; this feature is therefore essential, allowing users to select a domain of truth values appropriate to their application and automatically obtain the associated twist structure.

The Hasse diagrams of a lattice and their associated twist structures can be visualised directly in the interface. For example, Fig. 5 shows the rendering of the algebra **3** and its twist structure $\mathcal{NINE}$.

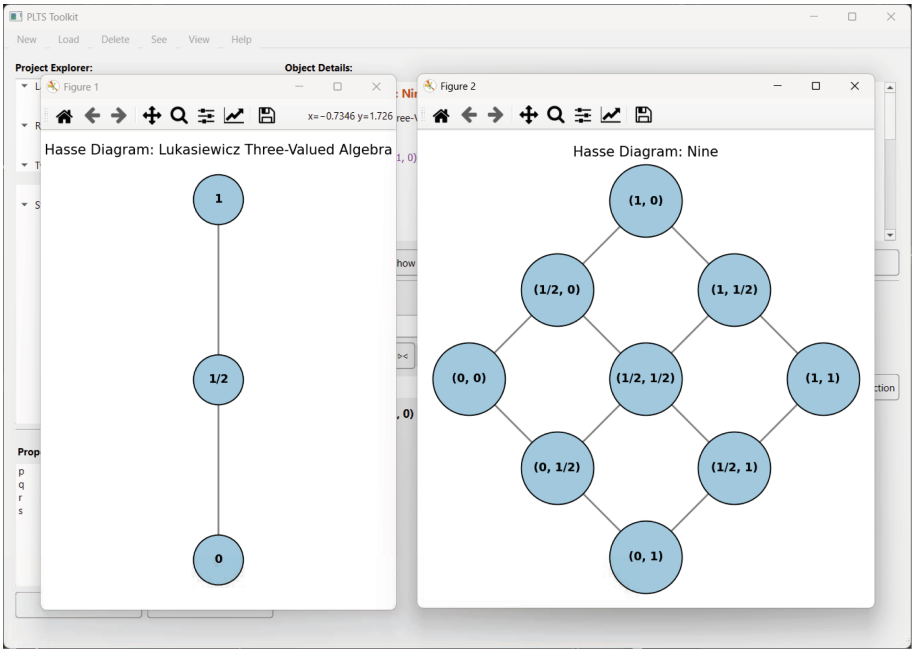

Fig. 5. Screenshot of the PLTS toolkit rendering of the Hasse diagram of **3** and $\mathcal{NINE}$.

At this stage, the toolkit supports complete lattices and their twist structures, enabling the construction of PLTS. More specifically, once a lattice and its twist structure are available, users may create states via the New menu by assigning, for each proposition, a value from the twist structure. Note that multiple states,

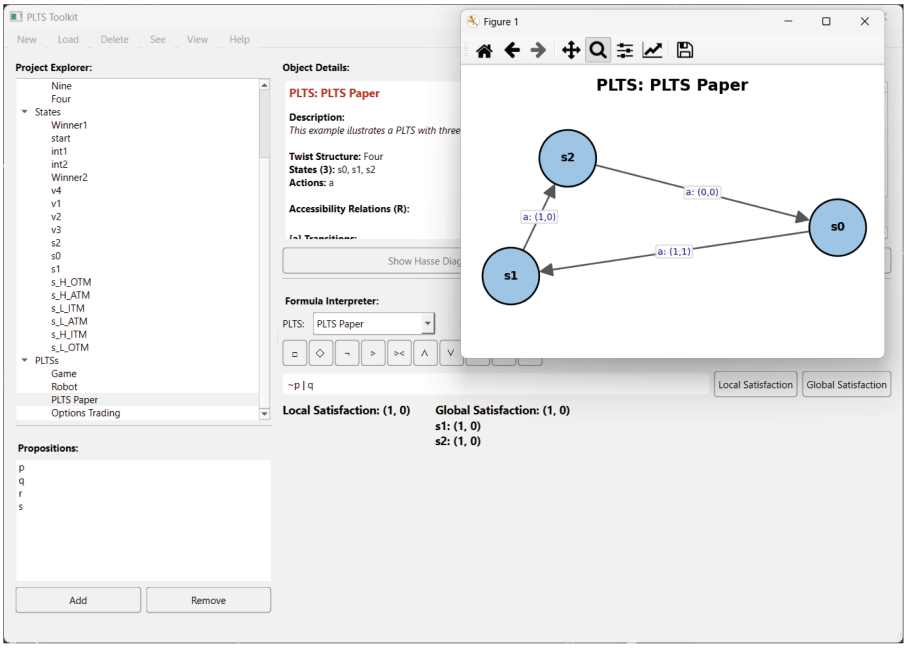

Fig. 6. Screenshot of the PLTS toolkit interpreting a formula for the PLTS in Fig. 3.

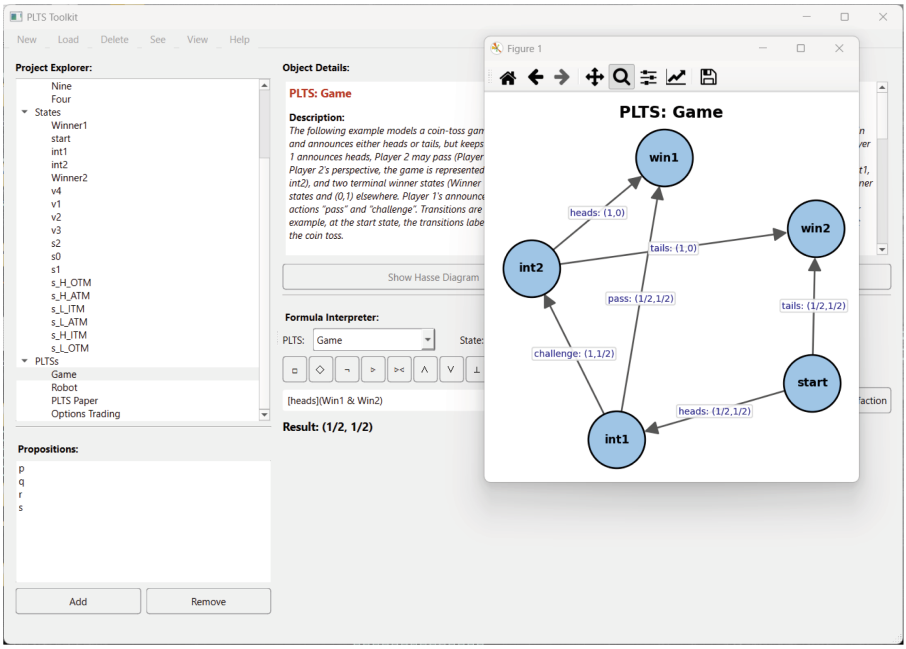

Fig. 7. Screenshot of the PLTS toolkit interpreting a formula for the PLTS in Fig. 9.

whose valuations take values from different twist structures, can be queued and created at once. A PLTS is then defined by selecting a set of states over a chosen twist structure, specifying action labels, and assigning a value in A^2 to each transition. When constructing a PLTS, users may also add a description of the PLTS, which appears alongside the model components in the object-details panel.

Figure 6 illustrates the graphical rendering produced by the toolkit for the PLTS of Fig. 3.

The toolkit also includes a built-in interpreter for the paraconsistent modal logic, recalled in Sect. 2, which validates formula syntax and reports errors. In the interpreter panel on the right, users may select a PLTS and a state, then input a formula using the available connectives. The Local satisfaction button then computes a pair representing evidence for and against the formula holding at the chosen state.

The Global satisfaction feature computes the value given by (1), and reports the value of local satisfaction for each state. Figures 6 illustrates local and global interpretation in the toolkit. Naturally, the cost of validity checking grows with the size of the PLTS.

Example 4. Recall the PLTS of Example 3 its rendering by the toolkit[2] is shown in Fig. 6. Using the interpreter, one can verify that the formula $\neg p \vee q$ is evaluated by the classical truth value $(1, 0)$ in every state. Moreover, the tool confirms that $[a]p \vee q$ evaluates to $(1, 0)$ at every state except s_1, where its value is $(0, 1)$, matching the observations in Example 3.

The remainder of this section presents illustrative case studies of the framework and toolkit on small-scale models. These examples can be loaded via the Load menu. Alongside a description of their components, each includes a brief descriptive summary in the object-details panel.

The following example, adapted from [7] which draws inspiration from [1], describes a robot navigating along a path while receiving possibly inconsistent information about obstacles from multiple sensors.

Example 5. In object detection, different sensors may provide conflicting information due to hardware limitations or environmental conditions. Combining multiple sensor is therefore a common practice to improve reliability and accuracy in obstacle detection and avoidance. We consider two groups of sensors: positive sensors, providing evidence that a path is clear, and negative sensors, providing evidence that it is obstructed.

The robot moves between four checkpoints represented as states of a PLTS. Transitions labelled by the action "move" represent movement between checkpoints and are weighted by values in the twist structure $\mathcal{NINE}$ of Example 2, combining the certainty from positive sensors that the path is clear with the certainty from negative sensors that it is obstructed. Certainty is a central aspect

[2] The corresponding model is available from the Load menu in the toolkit under the name "PLTS Paper".

of object detection, where the system not only identifies objects but also quantifies its confidence in the identification by, e.g. considering limitations in the precision of measurements and disregarding a sensor's output in the presence of environmental conditions that affect measurements.

The proposition Obst indicates whether a checkpoint contains an obstacle: it takes value $(1, 0)$ at v_3, the only obstructed checkpoint, and $(0, 1)$ at the remaining states. The resulting PLTS is shown in Fig. 8, where the obstructed state v_3 is highlighted[3].

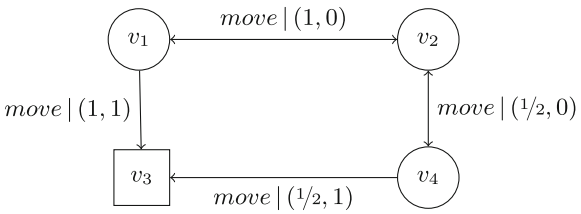

Fig. 8. PLTS representation of a robot navigating along a path with obstacles.

By using the logical interpreter, the formula Obst $\rightarrow$ [move]$\bot$, stating that the robot cannot move from an obstructed state, evaluates to $(1, 0)$ at every state. Indeed, the only state satisfying Obst is v_3, and there are no outgoing transitions from v_3, so the formula is evaluated by the classical truth value $(1, 0)$ at every state.

Consider also the formula $\langle$move$\rangle$Obst, indicating that it is possible to move to an obstructed checkpoint. For example, it follows that $(v_2 \models \langle\text{move}\rangle\text{Obst}) = (0, 1)$, since there is no transition from v_2 to v_3. Moreover, $(v_4 \models \langle\text{move}\rangle\text{Obst}) = (1/2, 1/2)$, reflecting uncertainty about reaching an obstructed state. This arises because the transition from v_4 to v_3 has the paraconsistent value $(1/2, 1)$, providing non-zero evidence both for and against its transversal.

The following example, adapted from [9], describes a simple coin-toss betting game between two players, a setting often considered in the literature of epistemic logic [19].

Example 6. Games frequently involve bluffing, incomplete information, and conflicting cues, requiring players to reason under uncertainty and possible contradiction. The paraconsistent framework employed in this paper is therefore suited to modelling such scenarios.

The game proceeds as follows. Player 1 always bets on heads, while Player 2 always bets on tails. Player 1 tosses a coin and announces either heads or tails while keeping the true outcome hidden. If Player 1 announces tails, Player 2 immediately wins. If heads is announced, Player 2 may either pass, in which

[3] This model is available via the Load menu in the toolkit under the name "Robot".

case Player 1 wins, or challenge, forcing Player 1 to reveal the true outcome; the player whose bet matches the outcome then wins.

From Player 2's perspective, the game can be represented by a PLTS with a starting state (*start*), intermediate states (*int*1 and *int*2), and winning states ($Winner1$ and $Winner2$). Propositions Win1 and Win2 take value $(1, 0)$ at their respective winner states and $(0, 1)$ elsewhere. Player 1's announcements correspond to actions "heads" and "tails", and Player 2's responses correspond to actions "pass" and "challenge". Transitions are weighted by values in the twist-structure $\mathcal{NINE}$ of Example 2 representing Player 2's evidence for and against each action.

Initially, Player 2 assumes the coin toss is fair, so both announcements "heads" and "tails" from the starting state have value $(1/2, 1/2)$. After a heads announcement, Player 2 evaluates several cues, such as Player 1's past behaviour and personal risk preference, to decide whether to pass or challenge. These considerations may lead to non-complementary degrees of evidence to each action. For instance, Player 2 may reason that:

(1) in the last two rounds Player 1 also announced heads, which seems unlikely;
(2) from prior rounds Player 1 is generally honest;
(3) Player 1 appeared nervous during the announcement;
(4) Player 2 is risk-seeking and inclined to challenge.

Consequently, Player 2 has strong evidence in favor of challenging, supported by arguments (1), (3), and (4), while argument (2) provides evidence against challenging. Hence, the transition labelled "challenge" is assigned the paraconsistent value $(1, 1/2)$.

Conversely, passing is moderately supported by argument (2) but weakened by the considerations in (1) and (3). Accordingly, the transition labelled "pass" receives the consistent value $(1/2, 1/2)$. The resulting PLTS is shown in Fig. 9.

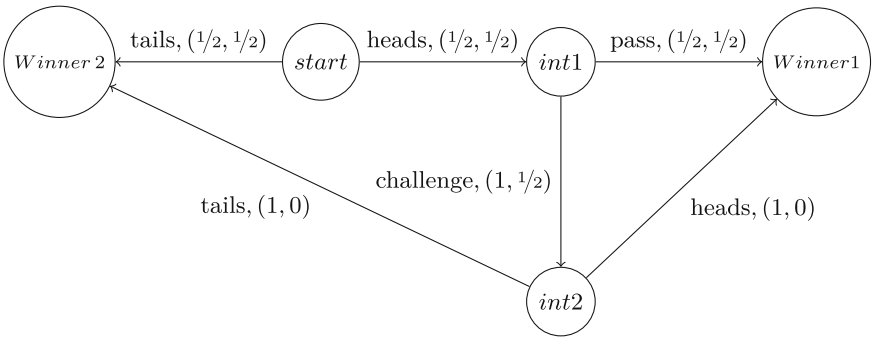

Fig. 9. PLTS representation of a coin-toss betting game.

By using the proposed toolkit[4] it is possible to construct and visualize this model as well as to evaluate specific formulae as depicted in Fig. 7.

One may then verify that the formula [tails]Win2 is evaluated to $(1, 0)$ at any state, i.e., whenever Player 1 announces tails, Player 2 wins. Another such formula is [tails]Win2 $\vee$ [heads]$\langle$pass$\rangle$Win1 $\vee$ [challenge]$\langle$heads$\rangle$Win1 indicating that tails guarantees Player 2's win, while heads followed either by passing or by a successful challenge leads to Player 1's win.

Naturally, invalid behaviours evaluate to $(0, 1)$, e.g. $\langle$tails$\rangle\langle$challenge$\rangle\top$, which would allow Player 2 to challenge after tails, and $\langle$pass$\rangle$Win2, stating that Player 2 wins after passing.

Finally,at the starting state the formula [heads](Win1 $\wedge$ Win2) evaluates to $(1/2, 1/2)$, indicating that after a heads announcement the winner is uncertain, since the subsequent choice between pass and challenge remains unresolved.

The following example illustrates the modelling of an options trading scenario using PLTS to reason about conflicting market signals.

Example 7. In the finance industry, an option on a stock is a financial derivative that grants the holder the right, but not the obligation, to buy or sell shares of that stock at a predetermined price, the strike price, within a specific timeframe [15]. American options allow exercise at any time before or on the expiration date, whereas European options can only be exercised on the expiration date.

To model options trading using a PLTS, one could integrate the Black-Scholes-Merton (BSM) model [15] to represent market signals and decision-making under uncertainty. In this framework, the BSM equation provides a theoretical value that serves as a benchmark against which market-defined states $(s_{ITM}, s_{ATM}, s_{OTM})$ are evaluated. Inconsistencies naturally arise when different indicators, such as a sudden volatility spike versus downward price dynamics, provide conflicting signals regarding the option's fair value. By adopting a paraconsistent approach, we can assign degrees of evidence for and against a specific transition, allowing the system to reason through such market contradictions.

The model's state space is defined by two temporal layers based on the time-to-expiration: High (s^H) and Low (s^L). Additionally, within each layer, states are categorised by their moneyness relative to the strike price (K) and current asset price (S), resulting in six possible states:

- s^H_{ITM} and s^L_{ITM} correspond to "In-the-Money" states further from and closer to the expiration date, respectively. In both cases it holds that $S > K$, meaning that the asset price exceeds the strike price and exercising the option would yield profit;
- s^H_{ATM} and s^L_{ATM} correspond to "At-the-Money" states further from and closer to the expiration date, respectively. In these states $S = K$, meaning the asset price equals the strike price and the option essentially breaks even;
- s^H_{OTM} and s^L_{OTM} correspond to "Out-of-the-Money" states further from and closer to the expiration date, respectively. In these states $S < K$, meaning

[4] This model is available via the Load menu in the toolkit under the name "Game".

the asset price is below the strike price and exercising the option would not be profitable.

The proposition profitable represents the evidence for and against an option being profitable at a given state. The valuation of this proposition across the state space may therefore be defined as follows:

- $V(s^H_{ITM}, \text{profitable}) = (1, 0)$, representing clear evidence that the option is profitable when it is in-the-money and still relatively far from expiration.
- $V(s^H_{ATM}, \text{profitable}) = (1/2, 1/2)$, representing balanced evidence when the option is at-the-money.
- $V(s^H_{OTM}, \text{profitable}) = (0, 1)$, representing clear evidence against profitability when the option is out-of-the-money.
- $V(s^L_{ITM}, \text{profitable}) = (1, 1/2)$, since the option is still in-the-money but proximity to expiration adds evidence against profitability, yielding a paraconsistent valuation.
- $V(s^L_{ATM}, \text{profitable}) = (0, 0)$, representing vague or unknown evidence when the option is close to expiration and exactly at the strike price.
- $V(s^L_{OTM}, \text{profitable}) = (0, 1)$, representing confirmed evidence against profitability.

Transitions between the defined states are driven by three categories of dynamics: market price dynamics, representing fluctuations in the underlying asset via price_up and price_down actions; temporal dynamics, using time_pass to model the transition from high to low expiration layers; and volatility dynamics, where vol_spike captures sudden increases in volatility.

For example, if there is a price increase, it becomes possible to transition from an out-of-the-money state to an at-the-money state. Similarly, if the price continues to rise, a transition from at-the-money to in-the-money may occur.

Temporal dynamics also influence the system. When time passes and the option is out-of-the-money, it is likely to remain out-of-the-money, since the remaining time for favourable price movements decreases. However, if time passes while the option is at-the-money, there may be some evidence against remaining in that state, reflecting the increased uncertainty as the expiration date approaches. Naturally, additional transitions could be considered depending on the particular market conditions being modelled.

The resulting PLTS[5] is shown in Fig. 10. By using the toolkit it is possible to construct and visualize this model as well as to evaluate specific formulae as depicted in Fig. 11.

One may then evaluate different formulae over the PLTS by using the built-in logical interpreter. For instance, as illustrated in Fig. 11, at state s^H_{OTM} the formula [price_up][price_up] profitable is evaluated as $(1, 0)$, indicating complete evidence that two successive price increases lead to a profitable state.

Additionally, at state s^H_{ITM} the formula [time_pass]⟨price_down⟩profitable is evaluated as $(0, 0)$, meaning that if the option is currently in-the-money and

[5] This model is available via the Load menu in the toolkit under the name "Options Trading".

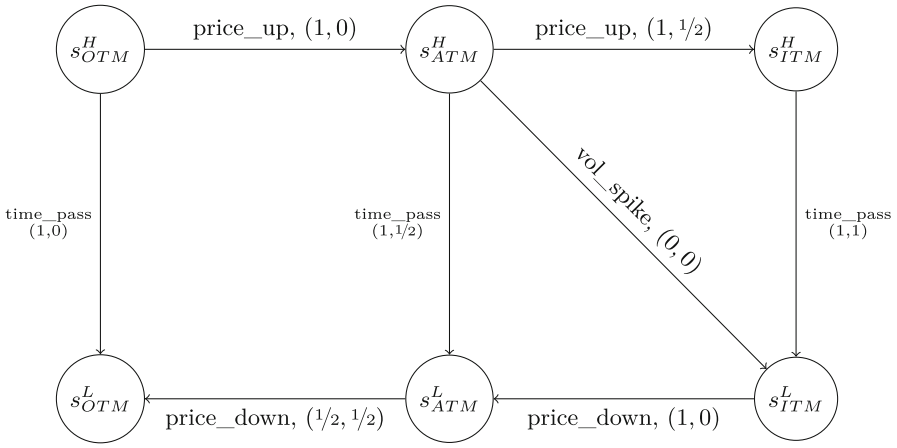

Fig. 10. PLTS representation of an options trading model, illustrating transitions between moneyness states across high and low time-to-expiration layers.

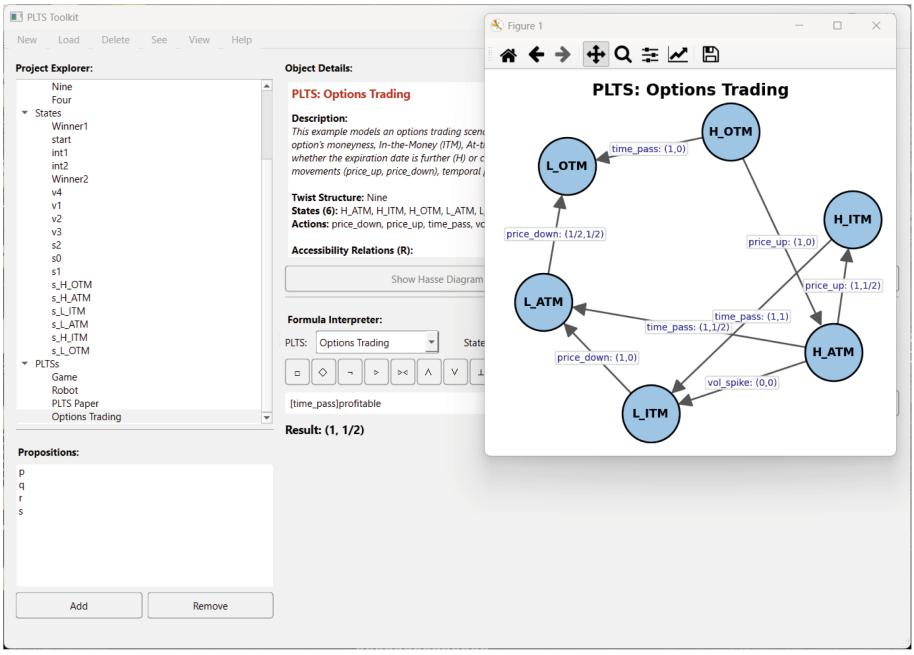

Fig. 11. Screenshot of the PLTS toolkit interpreting and checking the validity of formula for the PLTS in Fig. 10.

time passes, followed by a price decrease, it is uncertain whether the option will remain profitable.

Finally, still at state s^{H}_{ITM}, the formula $\langle \text{time_pass} \rangle$profitable is evaluated by the inconsistent value $(1, 1/2)$. This indicates that although there is evidence that the option may remain profitable as time passes, there is also some evidence against it. Such inconsistency reflects the fact that the option may approach the strike price as expiration nears, making profitability uncertain.

4 Conclusions

This paper contributes to an ongoing research agenda [7–10] aimed at developing frameworks and tools for representing and reasoning with inconsistent information. Within this line of work, PLTS together with their associated modal logic provide a framework for modelling such scenarios.

In this paper, we present the first steps towards a comprehensive toolkit for PLTS. Its main contribution is an interactive interface that enables users to construct finite arbitrary lattices and automatically generate their associated twist structures; to build and explore PLTS parametric over the chosen lattice; and to evaluate formulae of the paraconsistent modal logic and checks their validity over the resulting systems by using the built-in logical interpreter. Finally, we illustrate the applicability of the toolkit through illustrative case studies.

Several extensions remain for future work. In particular, the current implementation supports only finite lattices, whereas twist structures over infinite lattices also play a significant role in the literature [10,11]. Another promising direction is to allow twist structures to be formed from two possibly distinct lattices $\mathbf{L} = (L, \leqslant_{\mathbf{L}})$ and $\mathbf{R} = (R, \leqslant_{\mathbf{R}})$, yielding what are known as rectangular bilattices [11]. Such structures capture the fact that arguments in favor and against a statement can be evaluated on different scales. In contrast to existing approaches, we plan to directly combine values in $L \times R$ by considering interpretations between lattices, following [12].

Future versions will also implement enrichments of the modal logic with operators from dynamic logic, as introduced in [10], allowing the evaluation of properties such as deadlock avoidance and liveness. Moreover, features discussed in [7], such as checking paraconsistent bisimulations and incorporating a broader range of compositional constructions towards a full process algebra in the style of Nielsen and Winskel [22], are also planned for future versions.

Finally, this toolkit is expected to play a key role in forthcoming applications of this framework. In particular, it is expected to support a recently funded project (COMPETE2030-FEDER-00892000, Bansky – A paraconsistent inference engine for age-related macular degeneration) developed in collaboration with a medical research company. The project aims to model inconsistent datasets in degenerative-disease studies, where paraconsistent reasoning is particularly valuable for handling conflicting or incomplete information in the dataset.

References

1. Abe, J.M., Torres, C.R., Lambert-Torres, G., da Silva Filho, J.I., Martins, H.G.: Paraconsistent autonomous mobile robot emmy III. In: Lambert-Torres, G., Abe, J.M., da Silva Filho, J.I., Martins, H.G. (eds.) Advances in Technological Applications of Logical and Intelligent Systems, Selected Papers from the Sixth Congress on Logic Applied to Technology, LAPTEC 2007, Unisanta, Santa Cecilia University, Santos, Brazil, 21–23 November 2007. Frontiers in Artificial Intelligence and Applications, vol. 186, pp. 236–258. IOS Press (2007). https://doi.org/10.3233/978-1-58603-936-3-236
2. Akama, S. (ed.): Towards Paraconsistent Engineering, Intelligent Systems Reference Library, vol. 110. Springer (2016). https://doi.org/10.1007/978-3-319-40418-9
3. Barbosa, L.S., Madeira, A.: Capturing qubit decoherence through paraconsistent transition systems. In: Chiba, S., Cong, Y., Boix, E.G. (eds.) Companion Proceedings of the 7th International Conference on the Art, Science, and Engineering of Programming, Programming 2023, Tokyo, Japan, 13–17 March 2023, pp. 109–110. ACM (2023). https://doi.org/10.1145/3594671.3594689
4. Belnap, N.D.: A useful four-valued logic, pp. 5–37. Springer Netherlands, Dordrecht (1977). https://doi.org/10.1007/978-94-010-1161-7_2
5. Bou, F., Esteva, F., Godo, L., Rodríguez, R.O.: On the minimum many-valued modal logic over a finite residuated lattice. J. Logic Comput. **21**(5), 739–790 (2009). https://doi.org/10.1093/logcom/exp062
6. Chiara, M.L.D., Giuntini, R.: Paraconsistent ideas in quantum logic. Synth. **125**(1-2), 55–68 (2000). https://doi.org/10.1023/A:1005296018904
7. Cunha, J., Madeira, A., Barbosa, L.S.: Paraconsistent transition structures: compositional principles and a modal logic. Math. Struct. Comput. Sci. **35**, e14 (2025). https://doi.org/10.1017/S0960129525100170
8. Cunha, J., Madeira, A., Barbosa, L.S.: Stepwise development of paraconsistent processes. In: David, C., Sun, M. (eds.) Theoretical Aspects of Software Engineering - 17th International Symposium, TASE 2023, Bristol, UK, 4–6 July 2023, Proceedings. LNCS, vol. 13931, pp. 327–343. Springer (2023). https://doi.org/10.1007/978-3-031-35257-7_20
9. Cunha, J., Madeira, A., Barbosa, L.S.: Paraconsistent reactive graphs. In: Proença, J., Fervari, R., Martins, M.A., Kahle, R., Pluck, G. (eds.) Software Engineering and Formal Methods. SEFM 2024 Collocated Workshops - ReacTS 2024 and CIFMA 2024, Aveiro, Portugal, 4–5 November 2024, Revised Selected Papers. LNCS, vol. 15551, pp. 105–111. Springer (2024). https://doi.org/10.1007/978-3-031-94748-3_9
10. Cunha, J., Madeira, A., Barbosa, L.S.: Specification of paraconsistent transition systems. revisited. Sci. Comput. Program. **240**, 103196 (2025)
11. Deschrijver, G., Arieli, O., Cornelis, C., Kerre, E.E.: A bilattice-based framework for handling graded truth and imprecision. Int. J. Uncertain. Fuzziness Knowl. Based Syst. **15**(1), 13–41 (2007). https://doi.org/10.1142/S0218488507004352
12. Freire, A.R., Martins, M.A.: Modality across different logics. Log. J. IGPL **33**(3) (2025). https://doi.org/10.1093/JIGPAL/JZAE082
13. Ginsberg, M.L.: Multivalued logics: a uniform approach to reasoning in artificial intelligence. Comput. Intell. **4**, 265–316 (1988)
14. Hagberg, A., Swart, P., Chult, D.: Exploring network structure, dynamics, and function using networkx (2008). https://doi.org/10.25080/TCWV9851

15. Hull, J.: Options, Futures, and Other Derivatives. 11 edn., Pearson, Harlow, England (2021)
16. Hunter, J.D.: Matplotlib: a 2d graphics environment. Comput. Sci. Eng. **9**(3), 90–95 (2007). https://doi.org/10.1109/MCSE.2007.55
17. Kracht, M.: On extensions of intermediate logics by strong negation. J. Philos. Log. **27**(1), 49–73 (1998). https://doi.org/10.1023/A:1004222213212
18. Madeira, A., Neves, R., Martins, M.A.: An exercise on the generation of many-valued dynamic logics. J. Logical Algebraic Methods Program. **85**(5, Part 2), 1011–1037 (2016). https://doi.org/10.1016/j.jlamp.2016.03.004, https://www.sciencedirect.com/science/article/pii/S2352220816300256, articles dedicated to Prof. J. N. Oliveira on the occasion of his 60th birthday
19. Mousavi, M.R., Varshosaz, M.: Telling lies in process algebra. In: Pang, J., Zhang, C., He, J., Weng, J. (eds.) 2018 International Symposium on Theoretical Aspects of Software Engineering, TASE 2018, Guangzhou, China, 29–31 August 2018. pp. 116–123. IEEE Computer Society (2018). https://doi.org/10.1109/TASE.2018.00023
20. Priest, G.: Paraconsistency and dialetheism. In: Gabbay, D.M., Woods, J. (eds.) The Many Valued and Nonmonotonic Turn in Logic, Handbook of the History of Logic, vol. 8, pp. 129–204. Elsevier (2007). https://doi.org/10.1016/S1874-5857(07)80006-9
21. Beginning PyQt. Apress, Berkeley, CA (2022). https://doi.org/10.1007/978-1-4842-7999-1
22. Winskel, G., Nielsen, M.: Models for Concurrency, pp. 1–148. Oxford University Press, Inc., USA (1995)